

VAL CoPilot

Architecture & Process Flow
Synthetic Contract Intelligence Tool-Layer POC

Current runtime: Streamlit → Flask Cognitive Router → FastMCP Tools → Synthetic Gold
Data

Optional future: Azure AI Foundry Agent Service wrapping the same tools

Internal Engineering Document · Updated August 2026

1. Purpose & Scope

VAL CoPilot is a **synthetic contract intelligence tool-layer POC**. It proves that structured contract analytics can be exposed as MCP tools and consumed by a cognitive router that produces deterministic lifecycle workflows. It is intentionally **not** an Azure AI Foundry-first product demo, and it does not claim live Fabric / LinkSquares production connectivity.

In scope

- Synthetic Gold contract data via offline fixtures (`USE_OFFLINE MOCKS=true`)
- FastMCP tool layer: search, profiles, renewals, spend rollups, overlaps, risk explanations
- Cognitive router: Azure OpenAI + LangChain tool-calling, or offline keyword router
- Hard OOS guardrail for invoice/spend linkage requests
- Streamlit Validation UI for local demos
- Optional future Foundry Agent deployment path (same tools)

Out of scope for this POC

- Invoice / AP / spend-actuals data linkage (separate future POC)
- Live LinkSquares ingestion (future data source behind the same tools)
- Production Teams bot hosting as the primary demo path
- Claiming live Fabric SQL as the current validated demo path

2. Current Runtime Architecture

Four layers. Foundry is optional and not required for the current POC path.

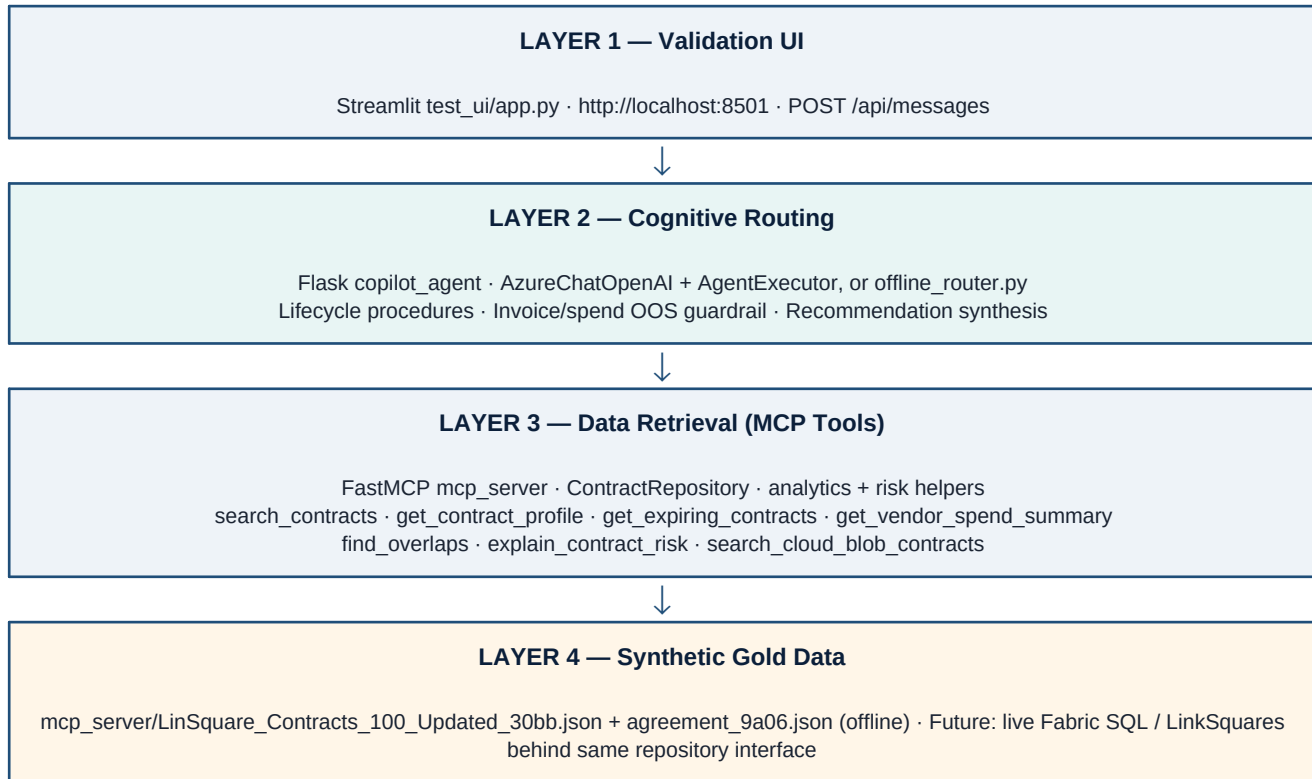


Figure 1 — Current POC runtime. Foundry Agent Service is an optional future host for Layer 2.

Request path

Step	Component	Action
1	Validation UI	User asks a contract question
2	Cognitive Router	OOS check → tool plan → MCP calls
3	MCP Tools	Repository + analytics/risk helpers
4	Synthetic Gold	Fixture rows (or future Fabric/LinkSquares)
5	Cognitive Router	Lifecycle Markdown + ## Recommendation
6	Validation UI	Render answer

3. MCP Tool Inventory

All tools are served by FastMCP. Structured contract tools are the primary POC surface. The optional Foundry ToolSet wraps the same structured surface (see Deployment notes).

Tool	Purpose	Notes
search_contracts	Keyword / field search	Vendor, category, status, ID
get_contract_profile	Full contract record	By ContractID
get_expiring_contracts	Renewal window	DaysAhead; notice fields
get_vendor_spend_summary	Committed value rollup	Not invoice actuals
find_overlaps	Same-vendor date overlaps	Risk helper
explain_contract_risk	Structured risk brief	facts / risks / gaps / action
search_cloud_blob_contracts	Blob keyword search	Optional Azure path

Hard OOS guardrail: invoice/spend-actuals questions never call tools. Exact reply:
Invoice/spend data is not part of this synthetic contract intelligence POC. This requires a separate data-linkage POC.

4. Repository Abstraction & Risk Helpers

ContractRepository isolates tool handlers from storage. Offline fixtures and future Fabric SQL share the same interface (`list_all`, `get_by_id`, `search`, `get_by_vendor`).

Risk helpers (`contract_risk.py`)

- **find_overlaps** — same-vendor Active/Pending contracts with overlapping date windows
- **explain_contract_risk** — returns `known_facts`, `computed_risks`, `missing_data`, `recommended_review_action`
- Demo seed: Microsoft CON-0024 $\cap$ CON-0029 overlap (`OverlapFlag=Yes`)

5. Lifecycle Procedures (Cognitive Layer)

Encoded in SYSTEM_PROMPT and mirrored by `offline_router.py`. Every answer ends with `## Recommendation`.

Procedure	Trigger	Output sections
Red-Flag Audit	Risk / compliance / audit	Risk Register · Missing Protections · ## Recommendation
Counter-Clause Drafting	Negotiate / rewrite	Weak Language · Counter-Clause · ## Recommendation
Financial Exposure	Liability / TCV / exposure	Exposure Summary · Liability Map · ## Recommendation
Renewal Strategy	Renew / expire / notice	Timeline · Decision Matrix · ## Recommendation
Comparative Analysis	Compare / vs / alternative	Side-by-Side · Trade-offs · ## Recommendation

6. Process Flow — Example Sequences

A. Happy path (renewal)

- UI → Router: “Which contracts expire in 90 days?”
- Router → get_expiring_contracts(90)
- MCP → repository filter by EndDate window
- Router → Renewal Strategy Sheet + ## Recommendation

B. Overlap / risk path

- UI → Router: “Any overlapping Microsoft contracts?”
- Router → find_overlaps / explain_contract_risk / search_contracts
- MCP → risk helpers over repository rows
- Router → Red-Flag Audit + ## Recommendation

C. OOS short-circuit

- UI → Router: “Show invoice totals for Microsoft”
- Router detects invoice/spend intent **before tools**
- Exact OOS message returned; no MCP calls

7. Local Runtime Topology

Process	Command	Port / URL
MCP server	python mcp_server/server.py	stdio (no HTTP port)
Cognitive router	python copilot_agent/app.py	http://localhost:3978
Validation UI	streamlit run test_ui/app.py	http://localhost:8501

Offline demo flags

USE_OFFLINE MOCKS=true
AZURE_OPENAI_FORCE_OFFLINE=true (optional – keyword router, no Azure OpenAI)

Verification

- python mcp_server/test_offline_mock.py
- python copilot_agent/test_poc_guard.py
- Demo script: docs/demo_script.md (8 deterministic questions)

8. Deployment Notes (Current vs Future)

Current POC path (required for demos)

- Run all three processes locally with offline fixtures
- No Foundry project required
- Do not commit `.env` (gitignored; use `.env.example`)

Optional future — Azure AI Foundry

- Script: `deploy_to_foundry.py` (optional — not required for POC demos)
- Guide: `docs/VAL_CoPilot_Azure_Foundry_Deployment_Guide.*`
- ToolSet wraps the structured MCP surface: `search_contracts`, `get_contract_profile`, `get_expiring_contracts`, `get_vendor_spend_summary`, `find_overlaps`, `explain_contract_risk`, `search_cloud_blob_contracts`
- Future host swap: Streamlit/Teams → Foundry Agent → same MCP tools → LinkSquares later

9. Evolution Roadmap

- **Now:** Synthetic Gold + MCP tools + cognitive router + Streamlit
- **Next:** Optional Foundry Agent host; extend ToolSet to full MCP inventory
- **Later:** LinkSquares as source behind ContractRepository (tool contracts unchanged)
- **Separate track:** Invoice / spend-actuals data-linkage POC

Related artifacts

- docs/VAL_CoPilot_Architecture_and_Process_Flow.pptx
- docs/demo_script.md
- docs/VAL_CoPilot_Azure_Foundry_Deployment_Guide.md
- mcp_server/server.py, contract_repository.py, contract_risk.py
- copilot_agent/agent.py, offline_router.py

End of document — Synthetic Contract Intelligence Tool-Layer POC · August 2026